

Rapport de stage Spécialité : Informatique

Développement d'un middleware léger pour l'orchestration d'APIs
CBS avec supervision et documentation intégrée

Réalisé par : Abker Ranim

Entreprise : IntechGeeks



Encadrant : Wael Saidi

Période de stage : 01/07/2025 - 31/08/2025

Table des matières

Introduction générale	1
1 Contexte général	3
1.1 Organisme d'accueil	4
1.1.1 Présentation d'IntechGeeks	4
1.1.2 Valeurs fondamentales	4
1.1.3 Structure organisationnelle	5
1.2 Présentation du sujet	5
1.2.1 Contexte général du projet	5
1.3 Problématique	6
1.4 Étude de l'existant	6
1.4.1 Limites de la solution	7
1.5 Solution proposée	7
1.6 Méthodologie adoptée	8
1.6.1 Formalisme de modélisation	8
1.6.2 Choix de la méthodologie de travail	8
1.6.3 Répartition des rôles	9
2 Analyse et Spécification des besoins	11
2.1 Spécification des acteurs	12
2.1.1 Identification des acteurs	12
2.2 Spécification des besoins	13
2.2.1 Besoins fonctionnels	13
2.2.2 Besoins non fonctionnels	16

3	Réalisation des tâches	19
3.1	Tâche 1 : Mise en route du CBS simulé via Docker	21
3.2	Tâche 2 : Développement du Middleware (APIs REST + appels CBS)	21
3.3	Tâche 3 : Intégration Swagger	21
3.4	Tâche 4 : Intégration OpenTelemetry	21
3.5	Tâche 5 : Développement du Dashboard React	21
	Conclusion générale et perspectives	22
	Références bibliographiques	23

Table des figures

1.1	Logo d’IntechGeeks	4
1.2	Cycle de vie SCRUM	10

Liste des tableaux

Introduction générale

La gestion du crédit est un processus critique pour les établissements financiers, englobant l'entrée en relation client, la préparation des dossiers, l'évaluation des risques, la prise de décision et la traçabilité. Dans de nombreux environnements opérationnels, ces activités sont réparties entre des outils hétérogènes ou reposent sur des plateformes low-code qui imposent des contraintes en matière de scalabilité, d'interopérabilité et de maintenabilité à long terme.

Une analyse de DigiLAF, la solution de gestion du crédit existante déployée chez l'entreprise d'accueil, a révélé que si elle numérise avec succès le workflow central du crédit, son architecture introduit des limitations en matière de scalabilité, de flexibilité vis-à-vis des fournisseurs et d'auditabilité qui deviennent critiques à grande échelle. Ces contraintes motivent la conception d'une plateforme sur mesure construite sur des technologies ouvertes et maintenables.

Pour répondre à ces besoins, ce projet propose **CrediWise**, une plateforme web sécurisée organisée autour de domaines métier indépendants. Le rapport présente l'analyse, la conception et l'implémentation de la plateforme en dix chapitres, chaque chapitre-sprint correspondant soit à la fondation DevOps, soit à un ou plusieurs microservices de l'architecture cible alignés sur une phase du workflow de demande de crédit :

- **Chapitre 1 : Contexte du Projet** — présente l'entreprise d'accueil, le contexte du projet, la solution existante, la solution proposée et la méthodologie.
- **Chapitre 2 : Analyse et spécification des besoins** — Ce chapitre aborde la définition des exigences fonctionnelles et non fonctionnelles. Nous y présentons en détail la planification des sprints ainsi que la gestion du backlog produit, tout en décrivant l'architecture technique de l'application.
- **Chapitre 3 : Sprint 1, Authentification et gestion de la clientèle** — présente la base de sécurité fondée sur Keycloak, ainsi que l'administration des agences, des gestionnaires et des rôles. Il introduit également la gestion des dossiers clients, les

données de référence et l'exposition du domaine client aux autres services via les mécanismes d'interconnexion entre services.

- **Chapitre 4 : Sprint 2, Service de Demande de Crédit** — détaille la création des demandes, la gestion des brouillons, le routage basé sur les produits et les transitions d'état du cycle de vie.
- **Chapitre 5 : Sprint 3, Service d'Analyse** — décrit le dossier d'analyse, le workflow par étapes et le calcul du scoring pour les produits éligibles à l'analyse.
- **Chapitre 6 : Sprint 4, Service Checklist** — présente la checklist pré-comité qui consolide les documents et les sections de vérification requis avant la phase comité.
- **Chapitre 7 : Sprint 5, Service d'Analyse des Risques** — décrit le formulaire d'analyse du risque de crédit (CRA), les modules d'avis et de Q&R, ainsi que la piste d'audit associée.
- **Chapitre 8 : Sprint 6, Services du comité de crédit et de décaissement** — présente le procès-verbal de décision du comité ainsi que le procès-verbal de décaissement, et la clôture du cycle de vie de la demande de crédit.
- **Chapitre 9 : Sprint 0, Fondation DevOps et Observabilité** — établit le pipeline CI/CD Azure DevOps, l'automatisation de build Maven, la génération d'images Docker, la topologie docker-compose de production avec le reverse proxy Nginx, l'instrumentation OpenTelemetry de chaque service backend, et la stack d'observabilité SigNoz utilisée pour le tracing distribué, les métriques et les logs centralisés.
- **Chapitre 10 :** —

Contexte général

Plan

1	Organisme d'accueil	4
2	Présentation du sujet	5
3	Problématique	6
4	Étude de l'existant	6
5	Solution proposée	7
6	Méthodologie adoptée	8

Introduction

Ce chapitre présente le contexte général du projet CrediWise. Il introduit l'entreprise d'accueil IntechGeeks, décrit le secteur de la microfinance en Tunisie, qui motive le projet, analyse les points forts et les limites de la solution existante DigiLAF, présente la plateforme CrediWise proposée et expose la méthodologie Scrum adoptée pour le développement.

1.1 Organisme d'accueil

1.1.1 Présentation d'IntechGeeks

IntechGeeks est une société d'ingénierie et d'intelligence artificielle au service de clients en Europe et en Afrique. Ses activités couvrent l'architecture logicielle cloud-native, l'intégration de systèmes et l'intelligence artificielle appliquée, avec un accent particulier sur la modernisation des systèmes d'information organisationnels. La figure ?? présente le logo de l'entreprise.



Figure 1.1 : Logo d'IntechGeeks

1.1.2 Valeurs fondamentales

Le travail de l'entreprise s'articule autour de principes qui guident la collaboration et la réalisation technique :

- **Rigueur technique** : Les systèmes livrés sont testés, documentés et instrumentés. Les tests automatisés, les pipelines reproductibles et l'observabilité sont considérés comme des critères essentiels de qualité du projet.

- **Immersion avant exécution** : Avant le début du développement, l'entreprise analyse le contexte opérationnel du client, les systèmes existants, les contraintes et les flux de travail, afin que la solution proposée corresponde aux besoins réels de l'entreprise.
- **Transfert de connaissances** : La livraison du projet comprend une documentation, l'explication des choix techniques et un accompagnement destiné à aider les équipes clientes à exploiter la solution livrée.
- **Transparence** : L'avancement du projet, les décisions techniques et les risques identifiés sont communiqués aux parties prenantes tout au long de la mission.

1.1.3 Structure organisationnelle

1.2 Présentation du sujet

Ce projet de fin d'études porte sur la conception et le développement de **CrediWise**, une plateforme web de gestion du crédit. L'objectif est de digitaliser et sécuriser l'ensemble du cycle de vie du crédit, de l'entrée en relation client jusqu'à la décision finale et la traçabilité.

La solution s'inscrit dans une architecture moderne basée sur des microservices et vise à réduire la dépendance aux solutions low-code existantes tout en améliorant la flexibilité, la scalabilité et la maintenabilité du système.

1.2.1 Contexte général du projet

Le secteur de la microfinance en Tunisie est supervisé par l'Autorité de Contrôle de la Microfinance (ACM). Les principaux acteurs — AKDI, Al Amal Microfinance, Zitouna Tamkeen, Advans Tunisie, Taysir Microfinance et Enda Tamweel, entre autres — jouent un rôle essentiel dans l'inclusion financière et le soutien des micro-entrepreneurs.

Malgré la croissance du secteur, la gestion du crédit reste un processus complexe, fortement documentaire et encore peu digitalisé en Tunisie et en Afrique en général.

Dans ce contexte, IntechGeeks a initié **CrediWise** comme module de gestion du crédit intégré à la plateforme KAYA, un middleware cloud-native permettant l'interconnexion de

systèmes financiers hétérogènes (Core Banking, CRM et ERP).

1.3 Problématique

Les solutions existantes de gestion du crédit souffrent de plusieurs limites, notamment en matière de scalabilité, de flexibilité et de maintenabilité.

Les processus restent souvent fragmentés, fortement dépendants de plateformes low-code propriétaires, ce qui entraîne :

- une limitation des performances sous forte charge,
- un coût élevé d'exploitation,
- une faible interopérabilité avec les systèmes externes,
- une traçabilité insuffisante des décisions,
- et une absence d'automatisation avancée (scoring, règles métier).

Ces contraintes rendent nécessaire la conception d'une solution moderne, modulaire et évolutive capable de supporter des processus de crédit complexes.

1.4 Étude de l'existant

La solution étudiée dans ce projet est **DigiLAF**, une application de gestion des crédits développée avec Microsoft Power Apps (Canvas App) et déployée dans plusieurs institutions financières en Tunisie et en Afrique.

Elle digitalise l'ensemble du cycle de crédit, de la création de la demande jusqu'au déblocage final, via un workflow basé sur le Business Process Flow.

Le processus comprend cinq étapes principales :

- **Analyse** : évaluation initiale et génération d'un rapport préliminaire.
- **Checklist** : vérification des documents requis.
- **Analyse des risques** : évaluation de la solvabilité et du risque.
- **Revue comité** : validation et décision du comité de crédit.

- **Visite de gestion** : contrôle terrain si nécessaire.

Power Automate est utilisé pour automatiser les transitions et les notifications, tandis que la gestion des accès repose sur les rôles de la Power Platform.

1.4.1 Limites de la solution

Malgré ses avantages, DigiLAF présente plusieurs limitations :

- scalabilité limitée,
- coût élevé (licences et connecteurs),
- dépendance forte à Microsoft,
- limitation de volume de données (2 000 enregistrements par requête),
- fragmentation du workflow,
- absence d'IA et de scoring automatique,
- faible observabilité et traçabilité,
- absence de portail client.

1.5 Solution proposée

Pour répondre à ces limites, ce projet propose **CrediWise**, une plateforme web de gestion du crédit basée sur une architecture modulaire et indépendante des solutions low-code.

La plateforme centralise la gestion des clients, des demandes de crédit, des analyses, des décisions et de la traçabilité dans une interface unifiée.

Fonctionnalités principales

- **Administration des utilisateurs et agences** : gestion des rôles et profils.
- **Gestion des clients** : centralisation des données clients.
- **Gestion des demandes de crédit** : création, brouillons, garanties et export PDF.
- **Routage par produit** : orientation automatique du workflow.
- **Analyse du crédit** : dossier structuré avec étapes d'évaluation.

- **Scoring et règles métier** : aide à la décision et traçabilité.
- **Modules de contrôle** : checklist, risques et comité de crédit.
- **Suivi du décaissement** : clôture du cycle de vie du crédit.

Sur le plan architectural, CrediWise repose sur une séparation claire entre frontend, backend, gestion d'identité (Keycloak) et bases de données. Cette architecture améliore la scalabilité, la maintenabilité et la traçabilité, tout en répondant aux limites de la solution existante.

1.6 Méthodologie adoptée

La réussite d'un projet informatique repose sur une méthodologie de travail structurée. Celle-ci permet d'organiser les activités, de respecter les délais et de garantir la qualité des livrables tout au long du cycle de développement.

1.6.1 Formalisme de modélisation

Pour la spécification et la conception du système, nous avons adopté le langage **UML** (Unified Modeling Language) [1]. Ce langage permet de représenter les besoins fonctionnels ainsi que l'architecture du système à travers plusieurs diagrammes :

- **Le diagramme de cas d'utilisation**, qui décrit les interactions entre les acteurs et le système.
- **Le diagramme de classes**, qui représente la structure statique du système, ses classes et leurs relations.
- **Le diagramme de séquence**, qui illustre les échanges entre les différents composants du système.

1.6.2 Choix de la méthodologie de travail

Afin d'assurer une gestion efficace du projet, nous avons adopté la méthodologie **Agile Scrum** [2]. Cette approche favorise une livraison progressive des fonctionnalités tout en

permettant une adaptation continue aux besoins.

Les principaux avantages de Scrum sont :

- Une meilleure visibilité sur l'avancement du projet.
- Une implication régulière des parties prenantes.
- Une priorisation des besoins selon leur importance.
- Une organisation du travail en sprints, facilitant le suivi du projet.

1.6.3 Répartition des rôles

Une équipe Scrum est composée du Product Owner, du Scrum Master et de l'équipe de développement. Ces rôles complémentaires collaborent afin d'assurer le bon déroulement du projet et la livraison régulière de fonctionnalités à valeur ajoutée.

1.6.3.1 Scrum Master

Le Scrum Master veille à l'application des principes Scrum et accompagne l'équipe dans leur mise en œuvre. Il facilite la communication et contribue à la résolution des obstacles rencontrés durant le projet.

1.6.3.2 Product Owner

Le Product Owner est responsable de la gestion du Product Backlog et de la priorisation des besoins. Il assure la liaison entre les parties prenantes et l'équipe de développement afin de garantir l'adéquation de la solution aux besoins métier.

1.6.3.3 Équipe de Développement

L'équipe de développement regroupe les membres chargés de concevoir, développer et tester la solution. À chaque sprint, elle produit un incrément fonctionnel respectant les critères de qualité définis.

1.6.3.4 Les événements de Scrum

La méthodologie Scrum repose sur plusieurs événements permettant de suivre l'avancement du projet :

- **Sprint Planning** : définition des objectifs et des tâches du sprint.
- **Daily Scrum** : réunion quotidienne de suivi et de coordination.
- **Sprint Review** : présentation et validation des fonctionnalités réalisées.
- **Sprint Retrospective** : identification des améliorations à apporter aux prochains sprints.
- **Backlog Refinement** : clarification et estimation des éléments du Product Backlog.

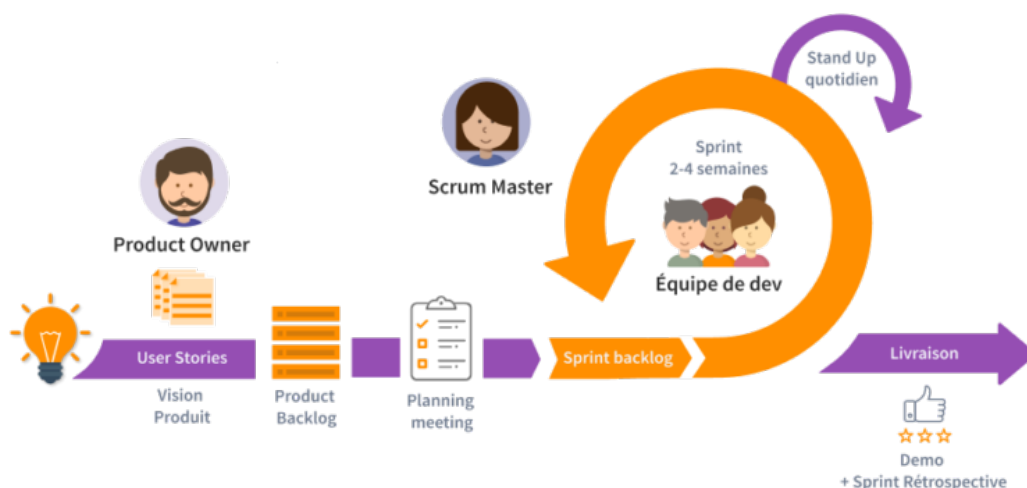


Figure 1.2 : Cycle de vie SCRUM

Conclusion

Ce chapitre débute par une présentation de l'organisme d'accueil. Par la suite, nous avons exposé une vue globale du projet, incluant la description de la solution actuelle, ses limites, la problématique identifiée, ainsi que la solution envisagée. Enfin, nous avons expliqué la méthodologie de gestion de projet adoptée. Le chapitre suivant se focalisera sur l'analyse et la spécification des besoins du projet.

Analyse et Spécification des besoins

Plan

1	Spécification des acteurs	12
2	Spécification des besoins	13

Introduction

Dans ce chapitre, nous identifions les caractéristiques du projet en analysant les besoins fonctionnels et non fonctionnels. Nous définissons également les acteurs et clarifions les exigences, qui sont ensuite représentées à travers un diagramme global de cas d'utilisation. Enfin, nous présentons l'organisation du développement en sprints.

2.1 Spécification des acteurs

2.1.1 Identification des acteurs

L'identification des acteurs permet de définir les périmètres d'accès à l'application. Dans CrediWise, les acteurs correspondent à des profils opérationnels définis dans Keycloak et propagés au frontend et au backend via un mécanisme d'autorisation basé sur les rôles.

Les acteurs suivants ont été identifiés :

- ▷ **Super Administrateur (SUPER_ADMIN)** : dispose du niveau d'accès le plus élevé. Il gère les agences, les comptes gestionnaires, les rôles, les règles d'affichage ainsi que les données administratives du système.
- ▷ **Utilisateur technique (TECH_USER)** : assure des tâches d'administration technique, principalement liées à la gestion des agences et des comptes gestionnaires, sans intervenir dans les processus métier de décision.
- ▷ **Chargé de relation client (CRO)** : consulte les dossiers clients, suit les demandes de crédit et complète les dossiers d'analyse lorsqu'ils lui sont affectés.
- ▷ **Agent front office (FRONT_OFFICE)** : assure l'accueil des clients, crée les fiches clients, prépare les demandes de crédit et soumet les brouillons pour traitement.
- ▷ **Décideur agence (BRANCH_DM)** : examine les demandes de crédit et les dossiers d'analyse au niveau de l'agence. Il participe aux décisions et aux transitions de statut selon les règles d'autorisation.
- ▷ **Décideur siège (HEAD_OFFICE_DM)** : consulte les dossiers nécessitant une

validation centrale et participe au processus de décision pour les demandes hors périmètre agence.

- ▷ **Analyste risque (RISK_ANALYST)** : analyse les demandes de crédit, les données d'évaluation et les résultats de scoring afin de contribuer à l'analyse des risques.
- ▷ **Auditeur (READ_ONLY)** : dispose d'un accès en lecture seule, utilisé pour les activités d'audit, de contrôle et de vérification.
- ▷ **Fournisseur d'identité (Keycloak)** : acteur système externe chargé de l'authentification, de la gestion des sessions et de l'émission des jetons. Il fournit également les rôles consommés par le frontend et le backend.

2.2 Spécification des besoins

Dans cette section, nous analysons les besoins fonctionnels et non fonctionnels du système, constituant une étape essentielle dans la définition de l'architecture du projet.

2.2.1 Besoins fonctionnels

▷ Authentification et sécurité

- Le système authentifie les utilisateurs via Keycloak et récupère les jetons d'accès à l'aide de l'adaptateur keycloak-js.
- Le frontend protège les routes en fonction des rôles de l'utilisateur, tandis que le backend applique les autorisations via l'annotation `@RolesAllowed`.
- Les jetons d'accès sont attachés aux requêtes API internes et renouvelés automatiquement avant expiration afin d'assurer la continuité des sessions.
- La déconnexion est gérée par Keycloak, garantissant la fermeture complète de la session au niveau du fournisseur d'identité.

▷ Gestion des utilisateurs et des agences

- Le service Manager permet de créer, modifier, désactiver et consulter les comptes gestionnaires.

- Le système prend en charge la gestion complète des agences (création, mise à jour, consultation et activation/désactivation).
- Les gestionnaires peuvent être associés à des agences, avec une cohérence assurée entre les rôles Keycloak et les données applicatives.
- Chaque utilisateur peut consulter et mettre à jour son profil selon les droits définis par l'API.

▷ Gestion des clients

- Le service Customer gère les fiches clients (personnes physiques et morales).
- Les données incluent l'identité, les coordonnées, l'agence, le type de compte, le segment, le secteur d'activité, le scoring et le cycle client.
- Le système permet la recherche de clients par identifiant ou coordonnées, ainsi que la consultation paginée des listes.
- Les données de référence (segments, secteurs, activités, types de compte, etc.) sont exposées pour garantir une saisie structurée.

▷ Gestion des demandes de crédit

- Le service Credit Request permet la création et la mise à jour des demandes de crédit au statut DRAFT.
- Une demande inclut les informations client, le montant, la durée, le produit, les garanties, les garants, la capacité de remboursement et les indicateurs de risque.
- Une fois soumise, la demande est routée selon le produit : les produits 101, 102 et 103 sont dirigés vers l'analyse, les autres vers la checklist pré-comité.
- Les acteurs autorisés peuvent mettre à jour le statut dans le cadre du cycle de vie du crédit.
- Une version PDF de la demande peut être générée pour archivage et documentation.

▷ Analyse du crédit et scoring

- Le service Analysis crée un dossier d'analyse pour chaque demande éligible.

- Le processus suit plusieurs étapes : informations client, objet du crédit, risques client, commerciaux et financiers, et données complémentaires.
- Les opérations de sauvegarde et de validation sont limitées aux profils autorisés.
- Un module de scoring calcule et enregistre le score de crédit associé à la demande.
- Les règles d’affichage sont versionnées afin de garantir la traçabilité des décisions.

▷ Liste de contrôle pré-comité

- Le service Checklist consolide les documents et vérifications nécessaires avant le passage en comité.
- La checklist est générée ou mise à jour à partir de la demande et enrichie via le service Customer.
- Elle couvre les documents, garanties, archivage, préparation au décaissement et contrôles front office/back office.

▷ Analyse du risque crédit

- Le service Risk Analysis gère le formulaire d’analyse du risque crédit (CRA).
- Le formulaire suit plusieurs statuts contrôlés : brouillon, en révision, validé, envoyé au comité, ou archivé.
- Les analystes peuvent compléter les sections d’avis et de questions-réponses.
- Un module d’audit enregistre l’historique des modifications.

▷ Décision du comité

- Le service Credit Committee enregistre et expose les décisions du comité de crédit.
- Les décisions sont initialisées à partir des services amont et mises à jour après validation.
- Elles influencent directement le cycle de vie de la demande (approbation, rejet ou décaissement).

▷ Décaissement

- Le service Disbursement enregistre les informations de décaissement pour les demandes approuvées.
- L'enregistrement est basé sur l'instantané de la demande de crédit.
- Une fois validé, le statut passe à « décaissé », et le compteur client est mis à jour via gRPC.

▷ Auditabilité et traçabilité

- Chaque service enregistre les métadonnées de création et de mise à jour des entités.
- La sécurité contrôle les accès aux routes API et aux services backend.
- Les utilisateurs en lecture seule peuvent consulter les données sans pouvoir les modifier, garantissant les besoins d'audit et de conformité.

2.2.2 Besoins non fonctionnels

▷ Performance

Le backend est implémenté avec Quarkus, offrant un temps de démarrage rapide et une faible consommation mémoire. Les API REST intègrent des mécanismes de pagination et de filtrage afin de limiter la taille des réponses. Le frontend, développé avec Vite et React, assure une navigation fluide et réactive en réduisant les rechargements de page.

▷ Scalabilité

L'architecture repose sur une séparation des principaux domaines métier en services indépendants : Manager, Customer, Credit Request et Analysis. Cette approche permet à chaque service, ainsi qu'à sa base de données associée, d'évoluer et d'être dimensionné indépendamment en fonction de la charge.

▷ **Fiabilité et disponibilité**

L'infrastructure locale est déployée via Docker Compose, incluant PostgreSQL, Keycloak et pgAdmin. La séparation des services et des bases de données limite l'impact des défaillances à un domaine spécifique et garantit la reproductibilité des environnements de développement.

▷ **Maintenabilité**

Chaque service backend est structuré comme un projet Maven indépendant avec son propre code, sa configuration, ses tests et ses migrations Flyway. Cette organisation réduit le couplage entre services et facilite la maintenance ainsi que l'évolution du système.

▷ **Conformité**

Le système traite des données personnelles et financières, ce qui impose des exigences strictes de traçabilité et de contrôle d'accès. L'utilisation d'instantanés immuables des demandes et de métadonnées d'audit garantit la cohérence des données et la conformité aux exigences réglementaires.

▷ **Sécurité**

L'authentification est centralisée via Keycloak. Les services backend valident les jetons JWT et extraient les rôles configurés pour appliquer les règles d'autorisation. Les contrôles d'accès sont appliqués au niveau des API et renforcés au niveau métier lorsque nécessaire. Les jetons ne sont pas stockés dans le stockage local du navigateur afin de réduire les risques de sécurité.

▷ **Ergonomie et expérience utilisateur**

Le frontend est une application monopage (SPA) développée avec React et TypeScript. Il propose une navigation basée sur les rôles, une gestion multilingue via i18next, une validation des formulaires et un système cohérent de gestion des erreurs afin d'améliorer l'expérience utilisateur.

Conclusion

Réalisation des tâches

Plan

1	Tâche 1 : Mise en route du CBS simulé via Docker	21
2	Tâche 2 : Développement du Middleware (APIs REST + appels CBS)	21
3	Tâche 3 : Intégration Swagger	21
4	Tâche 4 : Intégration OpenTelemetry	21
5	Tâche 5 : Développement du Dashboard React	21

3.1 Tâche 1 : Mise en route du CBS simulé via Docker

Introduction

Plan de réalisation

Besoins / Cas d'utilisation

Conception

Réalisation

Conclusion

3.2 Tâche 2 : Développement du Middleware (APIs REST + appels CBS)

Introduction

Plan de réalisation

Besoins / Cas d'utilisation

Conception

Réalisation

Conclusion

3.3 Tâche 3 : Intégration Swagger

Introduction

Plan de réalisation

Besoins / Cas d'utilisation

Conclusion générale et perspectives

Références bibliographiques

- [1] OBJECT MANAGEMENT GROUP. « Unified Modeling Language (UML) Specification. »
[Consulté le 2026]. adresse : <https://www.omg.org/spec/UML>
- [2] BOCASAY. « Méthode Scrum : quels sont ses bénéfices pour les développements web ? »
[Consulté le 2026]. adresse : <https://www.bocasay.com/fr/methode-scrum-benefices-developpements-web/>